

数据库系统原理期末大作业

实验7：NBA 数据分析智能体的设计与实现

项目	内容
课程	数据库系统原理
项目主题	基于 NBA 公开数据集的数据分析智能体
技术路线	Python + pandas + SQLite + LangChain + DeepSeek API + Streamlit
提交形式	RAR 压缩包，包含报告、代码、数据或数据获取说明、运行截图

角色	姓名	学号	任务分工
组员	邹传涛	37220232203970	环境搭建、报告整合、提交材料检查
组长	何后天	37220232203673	项目统筹、数据获取、pandas 清洗、SQLite 建库与验证查询
组员	彭泽喜	37220232203792	LangChain Data Agent、DeepSeek API 接入、Streamlit 可视化

说明：DeepSeek API Key 仅保存在本地 .env 文件中，报告和提交材料不包含真实密钥。

一、实验目的与总体要求

本实验围绕公开 NBA 篮球比赛数据集，设计并实现一个能够回答业务分析问题的数据智能体。项目覆盖真实数据集下载、清洗、关系数据库建模、SQL 查询、可视化展示和大模型辅助分析的完整流程。

- 使用 Python 和 pandas 完成数据读取、清洗、字段转换和统计概要生成。
- 使用 SQLite 建立关系数据库，提供建表、导入、索引和验证查询。
- 使用 LangChain 连接数据库，并结合 DeepSeek API 完成自然语言问题到 SQL 查询再到分析解释的流程。
- 使用 Streamlit 展示自然语言提问、SQL、查询结果、自然语言解释和图表。

二、开发环境与工具

类别	说明
操作系统	macOS，本地目录 /Users/whe/Desktop/BBB
Python	3.10.14
数据处理	pandas 2.3.3
数据库	SQLite，本地文件 db/nba_analysis.sqlite
Agent 框架	LangChain 1.3.11, langchain-openai 1.3.3
大模型 API	DeepSeek API, base_url=https://api.deepseek.com
界面与可视化	Streamlit 1.58.0, Plotly 6.8.0, Matplotlib 3.10.9
版本控制	Git，本项目已分阶段提交

三、数据集来源与字段说明

本项目选用 Kaggle 公开数据集 nathanlauga/nba-games。该数据集包含 2003 到 2022 年左右的 NBA 比赛、球队、球员、排名和球员单场技术统计数据，适合多表建模、关联查询和业务分析。

原始文件	用途	主要字段
games.csv	比赛事实表	比赛日期、赛季、主客队、比分、命中率、篮板、助攻、主队是否获胜等。
games_details.csv	球员单场统计	球员、球队、上场时间、投篮、三分、罚球、篮板、助攻、抢断、盖帽、得分等。
players.csv	球员信息	球员姓名、球员 ID、所属球队、赛季。
ranking.csv	球队排名	球队、日期、赛季、胜负场、胜率、主客场战绩等。
teams.csv	球队信息	球队 ID、缩写、城市、昵称、主场球馆、建队年份等。

四、数据预处理

数据预处理由 src/preprocess.py 完成。脚本读取 data/raw 下的 5 个 CSV 文件，统一字段名、转换类型、处理缺失值和重复记录，并输出清洗后的 CSV 到 data/processed。

- 字段名统一转为小写，便于 SQL 查询。
- 日期字段转为 YYYY-MM-DD 文本格式。

- 分数、命中率、篮板、助攻、抢断、盖帽等字段转为数值类型。
- 球员未上场记录保留，并使用 played 字段标记。
- 计数类技术统计缺失值按 0 处理；命中率类字段无法计算时保留为空。
- 按业务主键去除明显重复记录。

表	原始行数	清洗后行数	删除重复行数	清洗前缺失单元格	清洗后缺失单元格
teams	30	30	0	4	4
games	26651	26622	29	1188	1287
players	7228	7228	0	0	0
rankings	210342	210313	29	206352	206323
game_details	668628	668339	289	3804854	2268343

五、数据库设计与建库

数据库使用 SQLite，建库脚本为 src/build_database.py。该脚本会重新生成 db/nba_analysis.sqlite，创建表结构、导入清洗后的 CSV、建立索引，并输出验证查询结果。

表名	主键/唯一约束	说明
teams	team_id	球队维表，保存球队城市、昵称、缩写、主场、建队年份等。
games	game_id	比赛事实表，保存赛季、日期、主客队、比分、命中率和主队胜负。
players	player_id + team_id + season	球员赛季信息表。
rankings	自增 id	球队排名表，保存胜负场、胜率、主客场战绩。
player_game_stats	自增 id，唯一索引 game_id + player_id	球员单场技术统计表。

常用查询字段建立了索引，包括 games(season)、games(game_date)、games(home_team_id)、games(visitor_team_id)、player_game_stats(game_id)、player_game_stats(player_id)、player_game_stats(team_id)、rankings(team_id)、rankings(standings_date)。

table_name	row_count
teams	30
games	26622
players	7228
rankings	210313
player_game_stats	668339

六、Data Agent 设计与实现

数据智能体由 src/agent.py 实现。系统采用 LangChain 的 SQLiteDatabase 读取 SQLite schema，使用 langchain-openai 的 ChatOpenAI 以 OpenAI 兼容方式接入 DeepSeek API。

- 1 用户在 Streamlit 页面输入中文业务问题。

- 2 系统读取 SQLite 数据库表结构，并把 schema 和用户问题一起传给 DeepSeek。
- 3 DeepSeek 生成 SQLite 只读 SQL。
- 4 src/sql_safety.py 校验 SQL，只允许 SELECT 或 WITH，禁止危险语句。
- 5 SQL 校验通过后连接 SQLite 执行查询，返回真实结果表。
- 6 系统将问题、SQL 和结果样例传给 DeepSeek，生成中文业务分析结论。

项目	内容
SQL 生成提示词	要求模型只输出 SQLite SQL，不输出 Markdown 或解释；只能使用 SELECT/WITH；字段必须来自 schema。
结果解释提示词	要求模型只基于用户问题、SQL 和查询结果前 20 行作答，禁止编造事实。
人工修改内容	人工补充 SQL 安全检查、默认 LIMIT、错误提示和 Streamlit 展示逻辑。

七、Streamlit 界面与可视化

界面由 src/app.py 实现，页面显示数据库状态、示例问题、用户输入框、生成 SQL、查询结果、智能体解释和固定业务图表。固定图表不依赖大模型，能够直接用 SQLite 真实查询结果展示。

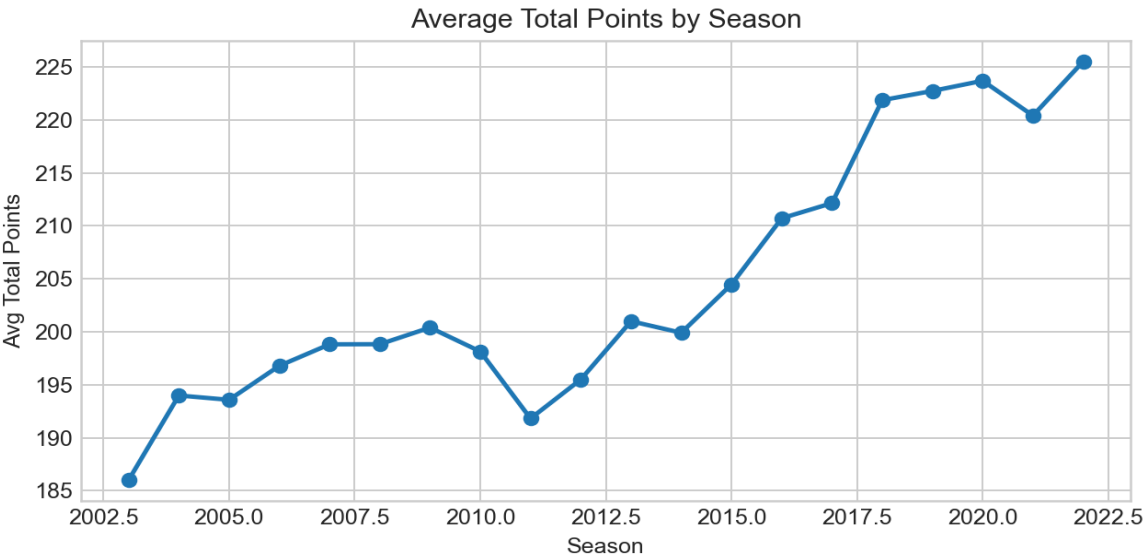


图 1 赛季场均总得分趋势。2003 赛季场均总得分为 186.00，后续赛季整体呈上升趋势。

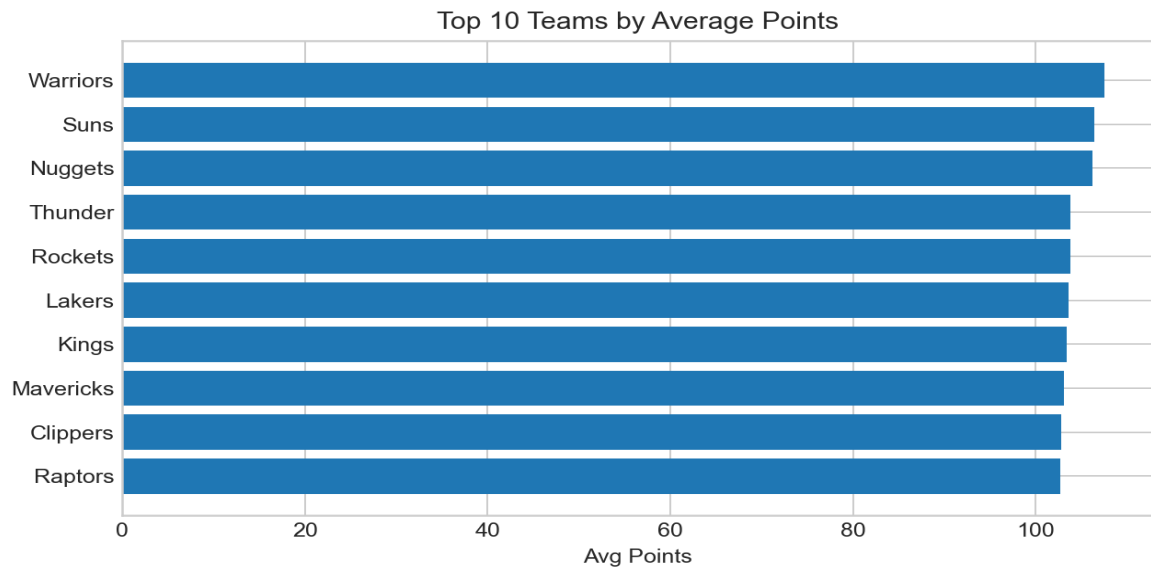


图 2 球队场均得分 Top 10。Warriors、Suns、Nuggets 位居前列。

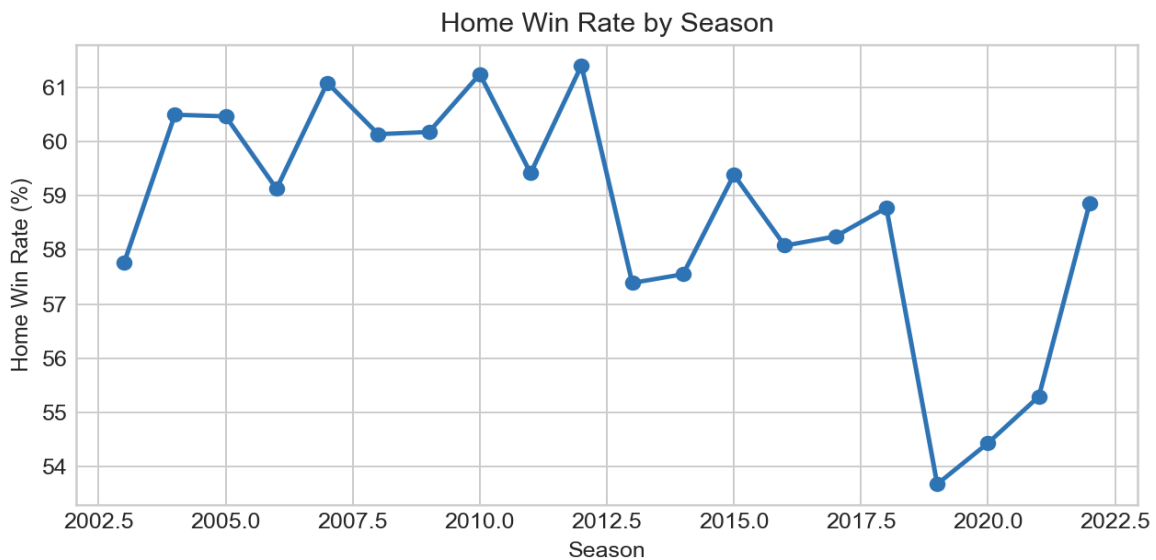


图 3 主队胜率随赛季变化。多数赛季主队胜率约在 57% 到 61% 之间。

八、业务分析案例

案例 1：每个赛季比赛数量

自然语言问题：每个赛季的比赛数量是多少？

智能体生成或等价执行 SQL：

```
SELECT season AS 赛季, COUNT(*) AS 比赛数量 FROM games GROUP BY season ORDER BY season LIMIT 8;
```

SQL 查询结果节选：

赛季	比赛数量
2003	1385
2004	1362
2005	1432
2006	1419
2007	1411
2008	1425
2009	1424
2010	1422

自然语言解释：2003 到 2010 年每个完整赛季大多在 1360 到 1430 场左右，2011 赛季比赛数量明显较少。

案例 2：每个赛季主队胜率

自然语言问题：每个赛季主队胜率是多少？

智能体生成或等价执行 SQL：

```
SELECT season AS 赛季, ROUND(AVG(home_team_wins)*100,2) AS 主队胜率百分比 FROM games WHERE home_team_wins IS NOT NULL
GROUP BY season ORDER BY season LIMIT 8;
```

SQL 查询结果节选：

赛季	主队胜率百分比
2003.00	57.76
2004.00	60.50
2005.00	60.47
2006.00	59.13
2007.00	61.09
2008.00	60.14
2009.00	60.18
2010.00	61.25

自然语言解释：主队胜率多数年份在 58% 到 61% 左右，说明主场优势在 NBA 比赛数据中较为明显。

案例 3：球队场均得分排名

自然语言问题：哪些球队场均得分最高？

智能体生成或等价执行 SQL：

```
SELECT t.nickname AS 球队, ROUND(AVG(team_points),2) AS 场均得分 FROM ... GROUP BY t.team_id,t.nickname ORDER BY 场均得分
DESC LIMIT 10;
```

SQL 查询结果节选：

球队	场均得分
Warriors	107.50
Suns	106.47
Nuggets	106.19
Thunder	103.79
Rockets	103.78
Lakers	103.56
Kings	103.44
Mavericks	103.14
Clippers	102.84
Raptors	102.67

自然语言解释：Warriors 以 107.50 的场均得分位居第一，Suns 和 Nuggets 紧随其后。

案例 4：球员单场得分 Top 10

自然语言问题：单场得分最高的 10 名球员是谁？

智能体生成或等价执行 SQL：

```
SELECT player_name AS 球员, team_abbreviation AS 球队, game_id AS 比赛ID, pts AS 得分 FROM player_game_stats WHERE pts IS NOT NULL ORDER BY pts DESC LIMIT 10;
```

SQL 查询结果节选：

球员	球队	比赛ID	得分
Kobe Bryant	LAL	20500591	81.00
Devin Booker	PHX	21601076	70.00
Kobe Bryant	LAL	20600977	65.00
Stephen Curry	GSW	22000092	62.00
Kobe Bryant	LAL	20500359	62.00
Tracy McGrady	ORL	20300927	62.00
Carmelo Anthony	NYK	21300640	62.00
Damian Lillard	POR	21901300	61.00
Damian Lillard	POR	21900652	61.00
Kobe Bryant	LAL	20800709	61.00

自然语言解释：Kobe Bryant 的 81 分位列第一，Devin Booker 的 70 分位列第二；Top 10 中 Kobe Bryant 多次出现。

案例 5：赛季场均总得分趋势

自然语言问题：不同赛季的场均总得分趋势如何变化？

智能体生成或等价执行 SQL：

```
SELECT season AS 赛季, ROUND(AVG(total_points),2) AS 场均总得分 FROM games WHERE total_points IS NOT NULL GROUP BY season ORDER BY season LIMIT 8;
```

SQL 查询结果节选：

赛季	场均总得分
2003.00	186.00
2004.00	193.99
2005.00	193.58
2006.00	196.79
2007.00	198.82
2008.00	198.83
2009.00	200.40
2010.00	198.14

自然语言解释：2003 到 2010 年场均总得分从 186.00 上升到 200 分左右，整体反映进攻产出提高。

九、大模型使用说明

项目	说明
模型名称	默认 deepseek-v4-flash，可通过 DEEPSEEK_MODEL 环境变量切换。
接口方式	OpenAI 兼容方式，base_url 为 https://api.deepseek.com。
使用环节	自然语言问题理解、SQL 生成、SQL 查询结果中文解释。
关键控制	temperature=0，提示词要求只输出 SQL，SQL 安全模块二次检查。
人工修改	人工编写数据清洗、建库、SQL 安全、错误处理、界面展示和报告内容。

十、运行步骤与复现说明

```
cd /Users/whe/Desktop/BBB
source .venv/bin/activate
python src/download_data.py
python src/preprocess.py
python src/build_database.py
python src/smoke_test.py
streamlit run src/app.py
```

如果需要启用 DeepSeek 智能问答，需要复制 .env.example 为 .env，并填写新的 DeepSeek API Key。真实 key 不写入报告和 Git。

十一、测试与验收结果

测试项	命令/方式	结果
依赖导入	python src/smoke_test.py	通过，关键依赖均可导入。
数据库检查	python src/smoke_test.py	通过，检测到 6 张表和 26622 场比赛。
SQL 安全检查	python src/smoke_test.py	通过，DROP TABLE 等危险语句被拒绝。
Streamlit 页面	streamlit run src/app.py	通过，本地健康检查返回 ok。
无 API Key 情况	python src/agent.py	通过，提示配置 DEEPSEEK_API_KEY，不直接崩溃。

十二、运行截图

以下截图用于证明项目可以在本地环境实际运行。图表截图已在正文“Streamlit 界面与可视化”部分展示，本节补充烟雾测试和 DeepSeek Agent 端到端查询链路截图。

```
Smoke Test Output
=====

Python package imports: ok
SQLite database: ok (6 tables, 26622 games)
SQL safety: ok
Smoke test passed.
```

图 4 本地烟雾测试运行截图。依赖导入、数据库检查和 SQL 安全检查均通过。

DeepSeek Agent Output

=====

Question: 每个赛季的比赛数量是多少？

SQL:

```
SELECT season, COUNT(game_id) AS game_count FROM games GROUP BY season  
LIMIT 100
```

Result:

season	game_count
2003	1385
2004	1362
2005	1432
2006	1419
2007	1411
2008	1425
2009	1424
2010	1422
2011	1104
2012	1420
2013	1427
2014	1418
2015	1416
2016	1405
2017	1382
2018	1378
2019	1241
2020	1220
2021	1389
2022	542

Analysis:

根据查询结果，2003年至2022年间，多数赛季的比赛数量在1380至1430场之间，但部分赛季出现显著异常：2011赛季仅有1104场（可能受停摆影响），2019和2020赛季分别降至1241场和1220场（可能与疫情相关），2022赛季仅542场（数据可能不完整）。整体来看，常规赛比赛量总体稳定，但受外部事件干扰会大幅波动。

图 5 DeepSeek Agent 自然语言查询运行截图。系统生成只读 SQL、执行查询并返回中文分析结论。

十三、总结

本实验完成了从 NBA 公开数据集下载、pandas 清洗、SQLite 建模、SQL 查询验证，到 LangChain + DeepSeek Data Agent 和 Streamlit 可视化展示的完整流程。系统能够根据中文业务问题生成只读 SQL，执行真实数据库查询，并基于查询结果生成中文解释。实验过程中重点保证了数据链路可复现、SQL 查询结果真实、API Key 不泄露、危险 SQL 被拦截，符合课程对数据处理、建库、查询、智能体和可视化展示的综合要求。